IVGS v5 — Test Implementation Plan

Spec Sections: §15.4 (80 % line coverage), §16.x (rate-limit thresholds)
Baseline: 153 tests passing · 63.6 % overall (2 651 / 4 167 lines)
Target: ≥ 80 % overall (3 334 lines covered → **+683 lines**)
Created: 2026-05-26 · **Workspace:** `/home/ubuntu/test_workspace/`

---

1. Executive Summary

The test suite currently covers **63.6 %** of application code. The deficit is concentrated in two layers:

Layer	Stmts	Covered	Gap to 80 %
services	1 325	504 (38.0 %)	**556 lines**
api	1 218	738 (60.6 %)	**236 lines**
middleware	182	154 (84.6 %)	0
models	562	546 (97.2 %)	0
schemas	654	629 (96.2 %)	0
core	92	80 (87.0 %)	0
scripts	134	0 (0.0 %)	**107 lines**

Services and API alone account for **~90 %** of the deficit. The plan attacks them in four phases ordered by coverage yield per test.

---

2. Coverage Gap Inventory

2.1 Worst Service Files (target: 80 % per-file)

Service File	Cov %	Stmts	Lines Needed
rollback_service.py	22.1	104	60
gpu_service.py	28.5	137	70
transcript_service.py	29.7	118	59
language_service.py	31.8	44	21
retention_service.py	33.3	75	35
story-board_service.py	34.2	79	36
check-point_service.py	34.3	67	30
job_service.py	35.1	37	16
quality_service.py	36.0	86	37
asset_service.py	38.5	96	39
dlq_service.py	42.9	105	39
project_service.py	43.4	136	49
prompt_service.py	48.7	117	36

2.2 Worst API Files

API File	Cov %	Stmts	Lines Needed
ws_logs.py	18.8	69	42
backup.py	43.3	120	44
manifests.py	42.0	119	45
alerts.py	41.2	17	7
languages.py	54.1	37	12
quotas.py	60.0	35	8
nodes.py	57.9	19	7
rollback.py	77.3	22	5

2.3 Untested Endpoints (no existing test file)

Endpoint	Verb	API File	Status
`/api/v1/backup/records`	GET	backup.py	**NO TESTS**
`/api/v1/backup/trigger`	POST	backup.py	**NO TESTS**
`/api/v1/backup/{id}/verify`	POST	backup.py	**NO TESTS**
`/api/v1/jobs/{id}/manifest`	GET	manifests.py	**NO TESTS**
`/api/v1/jobs/{id}/manifest/generate`	POST	manifests.py	**NO TESTS**
`/api/v1/jobs/{id}/manifest/lock`	POST	manifests.py	**NO TESTS**
`/api/v1/jobs/{id}/manifest/validate`	POST	manifests.py	**NO TESTS**
`/api/v1/alerts/webhook`	POST	alerts.py	**NO TESTS**
`/ws/nodes/{node_id}/logs`	WS	ws_logs.py	**NO TESTS**
`/ws/jobs/{job_id}/status`	WS	ws_logs.py	**NO TESTS**
`/api/v1/nodes`	GET	nodes.py	Partial
`/api/v1/languages`	GET/POST	languages.py	Partial
`/api/v1/quotas/{type}/{id}`	GET/PUT	quotas.py	Partial

2.4 Rate Limiting — Current Gaps (§16.3)

`rate_limit.py` is at 85.7 % (10 lines missing). The uncovered paths are:

Missing Lines	Path
45	`_get_client_ip` — `request.client is None` fallback
59-60	`_get_user_id_from_token` — decode failure catch
84-85	Lockout active → 429 response
111-115	Rate limit exceeded → 429 response
137-139	Lockout activation after N failures

Critical discovery: Redis mock `expire()` is a no-op. TTL-based window expiry **cannot be tested** without a real Redis or an enhanced mock.

---

3. Phased Implementation

Phase 1 — Rate Limiting Tests (§16.3) · ~30 new lines covered

New file: `tests/test_rate_limiting.py`

#	Test Function	What It Covers
1	`test_classify_login_request`	`_classify_request("/auth/login", "POST") == "login"`
2	`test_classify_job_trigger`	`_classify_request("/trigger", "POST") == "job_trigger"`
3	`test_classify_default`	`_classify_request("/projects", "POST") == "default"`
4	`test_get_client_ip_forwarded`	`X-Forwarded-For` header parsing
5	`test_get_client_ip_no_client`	`request.client is None` → `"0.0.0.0"` (line 45)
6	`test_crud_rate_limit_60_per_min`	POST 61 requests → 429 after 60 (lines 111-115)
7	`test_job_rate_limit_10_per_min`	POST `/trigger` 11 times → 429 after 10
8	`test_login_rate_limit_5_per_min`	POST `/auth/login` 6 times → 429 after 5
9	`test_login_lockout_after_10_failures`	10 × 401 → lockout key set → 429 (lines 84-85, 137-139)
10	`test_login_lockout_reset_on_success`	Successful login resets failure counter (line 145)
11	`test_get_requests_bypass_rate_limit`	GET requests skip middleware (line 74)
12	`test_non_api_paths_bypass`	Non-`/api/` paths skip middleware
13	`test_user_id_extraction_from_token`	Bearer token → user ID key
14	`test_user_id_extraction_failure`	Invalid token → falls back to IP (lines 59-60)
15	`test_retry_after_header_present`	429 response includes `Retry-After` header

Implementation notes:
- Import and test helper functions (`_classify_request` , `_get_client_ip`) as unit tests.
- For middleware integration tests, use `httpx.AsyncClient` with the app and manipulate the Redis mock's `incr()` counter to simulate window exhaustion.
- The lockout test requires the mock Redis to support `exists()` returning `True` after `set()` — verify mock supports this (current mock does).
- TTL expiry tests are **out of scope** (mock `expire()` is a no-op). Document this as a known limitation and recommend real-Redis integration tests in CI.

Projected yield: +30 lines in `rate_limit.py` → **92 %** (from 85.7 %)

Phase 2 — WebSocket Endpoint Tests (§13.4) · ~50 new lines covered

New file: `tests/test_websocket.py`

#	Test Function	What It Covers
1	`test_node_logs_unknown_node`	Unknown `node_id` → error JSON + close 1008 (lines 48-53)
2	`test_node_logs_accept_and_stream`	Known node → accept, mock subprocess output → JSON messages
3	`test_node_logs_with_service_filter`	`?service=api` → command includes service name (line 58)
4	`test_node_logs_without_service`	No service param → full compose logs (line 60)
5	`test_node_logs_client_disconnect`	Client disconnects mid-stream → graceful cleanup (line 87)
6	`test_node_logs_subprocess_error`	Subprocess raises → error JSON sent (lines 89-94)
7	`test_node_logs_process_terminates`	Stream ends → process terminated in finally (lines 96-97)
8	`test_job_status_receives_updates`	Pub/sub message → sent to client (lines 126-129)
9	`test_job_status_complete_closes`	Status “COMPLETE” → loop breaks (line 131)
10	`test_job_status_error_closes`	Status “ERROR” → loop breaks
11	`test_job_status_heartbeat`	No messages for 5s → heartbeat JSON (lines 134-135)
12	`test_job_status_client_disconnect`	Client disconnects → graceful cleanup (line 137)

Implementation notes:
- Use `httpx.ASGITransport` + `httpx.AsyncClient` for WebSocket testing, or `starlette.testclient.TestClient` with `with client.websocket_connect(...)`.
- Mock `asyncio.create_subprocess_shell` to return fake process with controlled stdout lines.
- Mock `redis.asyncio.from_url` to return a fake pub/sub that yields

controlled messages.
- The heartbeat test needs `asyncio.sleep` to be mocked (or use a short timeout).

Projected yield: +50 lines in `ws_logs.py` → **91 %** (from 18.8 %)

Phase 3 — Untested API Endpoints · ~120 new lines covered

3a. Backup API Tests

New file: `tests/test_backup_api.py`

#	Test Function	Lines Hit
1	`test_list_backup_records_empty`	GET /records → empty list (80-154)
2	`test_list_backup_records_with_data`	Seed records → paginated response
3	`test_list_backup_records_filter_type`	`?backup_type=full_db` filter
4	`test_list_backup_records_filter_status`	`?status_filter=completed` filter
5	`test_trigger_backup_admin_only`	POST /trigger → 200 (admin)
6	`test_trigger_backup_operator_denied`	POST /trigger → 403 (operator)
7	`test_verify_backup_completed`	POST /{id}/verify → 200
8	`test_verify_backup_not_found`	POST /{id}/verify → 404
9	`test_verify_backup_wrong_state`	Status=running → 422

Pre-requisite: Create `backup_records` table if not present (raw SQL `CREATE TABLE IF NOT EXISTS`). The table schema:

```
CREATE TABLE IF NOT EXISTS backup_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    backup_type VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'running',
    size_bytes BIGINT,
    started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ,
    verification_checksum VARCHAR(128),
    storage_path VARCHAR(500),
    error_message TEXT
);
```

Projected yield: +55 lines in `backup.py` → **89 %**

3b. Manifest API Tests

New file: `tests/test_manifest_api.py`

#	Test Function	Lines Hit
1	`test_get_manifest_not_found`	GET /{job_id}/manifest → 404
2	`test_get_manifest_success`	Seed manifest → 200
3	`test_generate_manifest`	POST /generate with scenes + assets
4	`test_generate_manifest_no_scenes`	→ 422
5	`test_generate_manifest_job_not_found`	→ 404
6	`test_lock_manifest`	POST /lock → status=locked
7	`test_lock_already_locked`	→ 409
8	`test_lock_not_found`	→ 404
9	`test_validate_manifest_valid`	All checksums match → valid=true
10	`test_validate_manifest_missing_asset`	→ errors list
11	`test_validate_manifest_checksum_mismatch`	→ errors list
12	`test_validate_manifest_not_found`	→ 404

Pre-requisite: Create `composition_manifests` table:

```
CREATE TABLE IF NOT EXISTS composition_manifests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id UUID NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'draft',
    timeline_json JSONB NOT NULL DEFAULT '{}',
    total_duration_ms INTEGER NOT NULL DEFAULT 0,
    scene_count INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    locked_at TIMESTAMPTZ
);
```

Projected yield: +65 lines in `manifests.py` → **97 %**

3c. Alerts, Nodes, Languages, Quotas

New file: `tests/test_alerts_api.py` (small — 3 tests)
Expand: existing test files or new ones for nodes, languages, quotas

File	Tests to Add	Lines Gained
test_alerts_api.py	webhook valid, webhook invalid, webhook auth	+10
test_nodes_api.py (new)	list nodes, get node detail, get node not found	+8
test_languages_api.py (new)	list variants, add variant, detect language	+17
test_quotas_api.py (new)	get quota, update quota, quota not found	+14

Projected yield: +49 lines across 4 API files

Phase 4 — Service Layer Expansion · ~450 new lines covered

This is the largest phase. Each service gets a dedicated unit-test file that tests the service class directly (with a real async DB session, no HTTP layer).

4a. High-Impact Services (ordered by lines needed)

Service	New Test File	Key Test Cases	Lines Gained
gpu_service.py	`test_gpu_service.py`	list/filter nodes, register upsert, drain (online/draining/offline), fleet util, _to_response with jobs	**+70**
rollback_service.py	`test_rollback_service.py`	create point (writes metadata.json), rollback_to (mock subprocess), list points, missing point → FileNotFoundError	**+60**
transcript_service.py	`test_transcript_service.py`	upload (mock SeaweedFS), list/get/update/delete, reorder (valid & invalid IDs), text extractors (txt, PDF mock, DOCX mock)	**+59**
project_service.py	`test_project_service.py`	create, list with filters, get, update, delete, trigger pipeline (with/without transcript)	**+49**
dlq_service.py	`test_dlq_service.py`	list messages with filters, get detail, replay, discard, analytics, bulk replay	**+39**
asset_service.py	`test_asset_service.py`	upload with dedup, list with filters, get metadata, delete, storage tier assignment	**+39**
quality_service.py	`test_quality_service.py`	get job quality (empty/with scores), list flagged (with joins), approve/reject state machine, reject+regeneration stub	**+37**

Service	New Test File	Key Test Cases	Lines Gained
prompt_service.py	`test_prompt_service.py`	create global, version increment, list/filter, resolve chain (global→project→scene), playground with Jinja2	**+36**
storyboard_service.py	`test_storyboard_service.py`	list scenes, update fields, reorder (valid/gaps/dupes), regenerate scene	**+36**
retention_service.py	`test_retention_service.py`	CRUD (create default clears others, name uniqueness), report (tier distribution, upcoming migrations)	**+35**
checkpoint_service.py	`test_checkpoint_service.py`	list (empty/with data), get stage, resume (failed job, stage ordering), clear	**+30**
language_service.py	`test_language_service.py`	list supported, add variant, detect language	**+21**
job_service.py	`test_job_service.py`	list jobs, create job, get/update status	**+16**

Total Phase 4 projected yield: ~527 lines (exceeds minimum needed)

4b. Service Test Pattern (template)

Each service test file follows this pattern:

```
"""Unit tests for {ServiceName}."""
import pytest
from uuid import uuid4
from sqlalchemy.ext.asyncio import AsyncSession

from app.services.{module} import {ServiceClass}

@pytest.fixture
async def service(db_session: AsyncSession):
    return {ServiceClass}(db_session)

class TestServiceCRUD:
    async def test_create(self, service):
        ...
    async def test_list(self, service):
        ...
    async def test_get_not_found(self, service):
        ...

class TestServiceBusinessLogic:
    async def test_specific_rule(self, service):
        ...
    async def test_edge_case(self, service):
        ...
```

Services that call external systems (SeaweedFS, Docker, subprocess) use `unittest.mock.AsyncMock` / `patch`:

```
@patch("app.services.transcript_service.seaweedfs_client")
async def test_upload(mock_sw, service):
    mock_sw.upload = AsyncMock(return_value={"fid": "1,abc"})
    ...
```

Phase 5 — Scripts Coverage · ~80 new lines covered

New file: `tests/test_scripts.py`

Script	Stmts	Test Cases
`create_admin.py`	42	Import + call with mock DB → admin user created
`seed_fallback_policies.py`	48	Import + call → policies seeded, idempotent re-run
`seed_prompts.py`	44	Import + call → prompts seeded, idempotent re-run

Implementation notes:
- Scripts use `asyncio.run()` at module level — wrap in `if __name__ == "__main__":` guard if not already present, or import

the inner function directly.
- Mock `AsyncSession` and verify SQL calls.

Projected yield: +80 lines → scripts at ~60 % (sufficient for overall 80 %)

---

4. Coverage Projection

Phase	New Lines Covered	Cumulative Total	Overall %
Baseline	—	2 651	63.6 %
Phase 1: Rate Limiting	+30	2 681	64.3 %
Phase 2: WebSocket	+50	2 731	65.5 %
Phase 3: API Endpoints	+120	2 851	68.4 %
Phase 4: Services	+527	3 378	**81.1 %**
Phase 5: Scripts	+80	3 458	**83.0 %**

Phase 4 alone crosses the 80 % threshold. Phase 5 provides margin.

---

5. Priority & Execution Order

```
Phase 1 (Rate Limiting)     ▓▓▓▓▓▒▒▒▒▒  ~2 hours   ← Spec §16.3 compliance
Phase 2 (WebSocket)         ▓▓▓▓▓▓▓▒▒▒  ~3 hours   ← Spec §13.4 compliance
Phase 3 (API Endpoints)     ▓▓▓▓▓▓▓▓▓▓  ~4 hours   ← Untested endpoints
Phase 4 (Services)          ▓▓▓▓▓▓▓▓▓▓  ~6 hours   ← Coverage bulk
Phase 5 (Scripts)           ▓▓▓▒▒▒▒▒▒▒  ~1 hour    ← Coverage margin
                                        ──────────
                            Total:      ~16 hours
```

If time-constrained, complete Phases 1-4 only. That reaches 81.1 % and covers all spec-mandated categories (rate limiting, WebSocket, services).

---

6. Test Infrastructure Requirements

6.1 Database Tables Needed

The following tables must exist for new tests. They are created in `conftest.py` via raw SQL `CREATE TABLE IF NOT EXISTS`:

Table	Needed By
`backup_records`	Phase 3a (backup API)
`composition_manifests`	Phase 3b (manifest API)
`alembic_version`	Phase 4 (rollback service)

6.2 Mock Enhancements

Mock	Enhancement	Needed By
Redis mock	`exists()` must return `True` after `set()`	Phase 1 (lockout)
Redis mock	`pubsub()` must support `subscribe/get_message`	Phase 2 (WS job status)
SeaweedFS mock	Already present in conftest	Phase 4 (transcript)
`asyncio.create_subprocess_shell`	Return mock process	Phase 2 (WS logs), Phase 4 (rollback)

6.3 Shared Fixtures Needed

```python
# conftest.py additions
@pytest.fixture
async def backup_record(db_session) -> str:
    """Seed a completed backup record and return its ID."""

@pytest.fixture
async def manifest_with_scenes(db_session, project_id, job_id) -> str:
    """Seed a composition manifest with timeline JSON and return manifest ID."""

@pytest.fixture
async def redis_with_pubsub():
    """Enhanced Redis mock supporting pub/sub."""
```

---

7. Risk Register

Risk	Impact	Mitigation
Redis mock `expire()` is a no-op	TTL-based window expiry untestable	Document as known limitation; recommend real-Redis in CI
Scripts have module-level `asyncio.run()`	Import fails in test context	Wrap in `__main__` guard or mock `asyncio.run`
`backup.py` line 299 references undefined `exc` variable	Runtime bug in error handler	Fix before testing (rename `_exc` → `exc`)
`manifests.py` line 85-92 has dead code (broken select)	Runtime error if reached	Tests will reveal; fix inline
Parallel test execution breaks DB	False failures	Keep `pytest -p no:xdist` (serial execution)
`composition_manifests` table may not exist	Test setup failure	Add to conftest `CREATE TABLE IF NOT EXISTS`

---

8. Definition of Done

Per-Phase Gate

- [ ] All new tests pass (`pytest -v`)
- [ ] No existing tests regressed (153 still passing)
- [ ] Coverage delta matches projection (±5 %)
- [ ] Git committed with descriptive message

Final Gate (all phases)

- [ ] Overall coverage ≥ 80 % via `pytest --cov`
- [ ] Rate limiting: all 3 tiers tested (60/10/5 req/min)
- [ ] Rate limiting: lockout after 10 failures tested
- [ ] WebSocket: both endpoints tested (node logs + job status)
- [ ] All untested endpoints have at least happy-path + 404 tests
- [ ] Service layer: all 13 services have dedicated test files
- [ ] No suppression hooks remain (already verified: 0 needed)
- [ ] `git log --oneline` shows clean commit history

---

9. File Manifest

New test files to be created:

File	Phase	Est. Tests
`tests/test_rate_limiting.py`	1	15
`tests/test_websocket.py`	2	12
`tests/test_backup_api.py`	3	9
`tests/test_manifest_api.py`	3	12
`tests/test_alerts_api.py`	3	3
`tests/test_nodes_api.py`	3	3
`tests/test_languages_api.py`	3	3
`tests/test_quotas_api.py`	3	3
`tests/test_gpu_service.py`	4	10
`tests/test_rollback_service.py`	4	6
`tests/test_transcript_service.py`	4	10
`tests/test_project_service.py`	4	8
`tests/test_dlq_service.py`	4	8
`tests/test_asset_service.py`	4	7
`tests/test_quality_service.py`	4	7
`tests/test_prompt_service.py`	4	7
`tests/test_storyboard_service.py`	4	6
`tests/test_retention_service.py`	4	6
`tests/test_checkpoint_service.py`	4	6
`tests/test_language_service.py`	4	4

File	Phase	Est. Tests
`tests/test_job_service.py`	4	4
`tests/test_scripts.py`	5	6
Total		**~155 new tests**

Combined with 153 existing → **~308 total tests**.

---

10. Known Bugs Found During Analysis

These were discovered during coverage gap analysis and should be fixed before or during test implementation:

1. **`backup.py` line 299:** References `exc` but the exception is bound as `_exc` (noqa suppressed). Will cause `NameError` at runtime.
   Fix: Change `_exc` → `exc` on line 292.

2. **`manifests.py` lines 85-92:** Dead code using broken `select("*")` pattern. The actual query starts on line 96. Lines 85-92 are unreachable dead code that would fail if reached.
   Fix: Remove lines 85-92.

3. **Redis mock TTL no-op:** `expire()` silently succeeds but never expires keys. Rate limit window reset is untestable.
   Risk: Low (server uses real Redis in production).

End of plan. Execute phases sequentially in `/home/ubuntu/test_workspace/`.
Commit after each phase with message `test: phase N — <description>`.